基于像素到底有什么不好？😥

1.对原图像逐像素处理，处理的就是单个像素之间的差距，如题面所说没有考虑到空间关系，而预训练模型中间层特征图中的每一块像素实际上都代表着原图像某一块区域的所有特征，这样就会将图像的空间关系纳入考虑，损失计算也更偏向于整体的内容差异而不是单个像素的差异。

2.这个训练好的网络可以判断图像的内容，将图像转化为一个象征着图像内容、结构的特征图。

3.

①补全 `perceptual_loss` 代码：

```
import torch as th
import torch.nn as nn
from torchvision.models import vgg19, VGG19_Weights

class PerceptualLoss(nn.Module):
    def __init__(self):
        super(PerceptualLoss, self).__init__()
        vgg = vgg19(weights = VGG19_Weights.IMAGENET1K_V1).features
        self.slice = vgg[:35]
        self.slice.eval()
        for p in self.slice.parameters():
            p.requires_grad = False
        self.mean = th.tensor([0.485, 0.456, 0.406]).view(1, 3, 1, 1)
        self.std = th.tensor([0.229, 0.224, 0.225]).view(1, 3, 1, 1)
        self.criterion = nn.L1Loss()

    def forward(self, generated, ground_truth):
        gen_norm = (generated - self.mean) / self.std
        gt_norm = (ground_truth - self.mean) / self.std

        feat_gen = self.slice(gen_norm)
        feat_gt = self.slice(gt_norm)

        return self.criterion(feat_gen, feat_gt)
```

②使用命令 `python main.py --data_dir "./data/Rain200L" --model unet --mode train --epochs 50 --use_perceptual_loss` 将 `perceptual_loss` 作为损失的一部分训练 `unet` 模型。

测试结果：

```
(derain) pigeon@LAPTOP-GARN0974:~/ml-week/Image-Deraining-Recruitment-Task$ python main.py  --data_dir "./data/Rain200L" --model unet --mode test  --checkpoint
"checkpoints/best_unet_perceptual.pth.tar"
Using device: cpu
Setting up [LPIPS] perceptual loss: trunk [alex], v[0.1], spatial [off]
/home/pigeon/miniconda3/envs/derain/lib/python3.10/site-packages/torchvision/models/_utils.py:208: UserWarning: The parameter 'pretrained' is deprecated since 0
.13 and may be removed in the future, please use 'weights' instead.
  warnings.warn(
/home/pigeon/miniconda3/envs/derain/lib/python3.10/site-packages/torchvision/models/_utils.py:223: UserWarning: Arguments other than a weight enum or `None` for
 'weights' are deprecated since 0.13 and may be removed in the future. The current behavior is equivalent to passing `weights=AlexNet_Weights.IMAGENET1K_V1`. Yo
u can also use `weights=AlexNet_Weights.DEFAULT` to get the most up-to-date weights.
  warnings.warn(msg)
Loading model from: /home/pigeon/miniconda3/envs/derain/lib/python3.10/site-packages/lpips/weights/v0.1/alex.pth
/home/pigeon/miniconda3/envs/derain/lib/python3.10/site-packages/lpips/lpips.py:107: FutureWarning: You are using `torch.load` with `weights_only=False` (the cu
rrent default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code durin
g unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for `weigh
ts_only` will be flipped to `True`. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded
 via this mode unless they are explicitly allowlisted by the user via `torch.serialization.add_safe_globals`. We recommend you start setting `weights_only=True`
 for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.
  self.load_state_dict(torch.load(model_path, map_location='cpu'), strict=False)
/home/pigeon/ml-week/Image-Deraining-Recruitment-Task/main.py:136: FutureWarning: You are using `torch.load` with `weights_only=False` (the current default valu
e), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See
https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for `weights_only` will be f
lipped to `True`. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unl
ess they are explicitly allowlisted by the user via `torch.serialization.add_safe_globals`. We recommend you start setting `weights_only=True` for any use case
where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.
  load_checkpoint(torch.load(args.checkpoint, map_location=device), model, optimizer)
=> Loading checkpoint
EVAL => Avg PSNR: 30.1340 | Avg SSIM: 0.8864 | Avg LPIPS: 0.0489
```

你知道什么是向量吗？😊

1.向量的定义是带有数值和方向的量，在机器学习中就可以表示为由对象的不同特征表现出的列表，在此基础上，向量的维度就可以理解为特征的数量

2.内积可以表示为一个向量映射到另一个向量上的模长和另一个向量的模长之积，也可以表示为两个向量每个维度对应的数值之积的和。而在线性代数中，矩阵乘法本身就可以表示为每一个行向量与每一个列向量的内积。

3.在机器学习中，向量的内积可以用来表示两个向量的相关性，还可以用于计算gram矩阵来衡量每两个不同特征的同步程度。余弦相似度就是将两个向量转化为方向不变的单位向量后的内积，其实就是内积几何定义中的角度 $\cos\theta$ ，可以用来衡量两个向量在方向上的接近程度，余弦相似度的范围为 $[-1,1]$ ，越接近1表明两个向量的方向越接近。

改造感知损失函数，使用特征做对比😌

1.先在 `get_args` 函数里添加 `use_contrastive_loss` 和 `lambda_contrastive` 两个参数，分别表示是否启用 `easy_contrastive_loss` 以及对应的权重是多少，默认会使用这个损失函数，且考虑平衡损失函数之间的量级将权重设置为0.1（与之相对的，`pixel_loss` 的权重是1.0，`perceptual_loss` 的权重是0.1）。

```
parser.add_argument("--use_contrastive_loss", action="store_true", help="Use easy
contrastive loss")
parser.add_argument("--lambda_contrastive", type=float, default=0.1, help="Weight
for contrastive loss")
```

然后再 `train_one_epoch` 中修改相关代码，当 `args.use_contrastive_loss` 为 `True` 时计入 `contrastive_loss` 这一损失（并在函数参数中加上 `contrastive_loss_fn`），另外考虑将这一损失加入进度条中，考虑重置这一部分的代码，用一个字典 `postfix` 装入要计入总loss的损失函数，最终只需要用一次 `loop.set_postfix(postfix)` 即可。

```
rainy, clean = rainy.to(device), clean.to(device)
derained = model(rainy)
pixel_loss = args.lambda_pixel * pixel_loss_fn(derained, clean)
total_loss = pixel_loss
postfix = {"pixel_loss": f"{pixel_loss.item():.4f}"}
```

```
if args.use_perceptual_loss:
    p_loss = args.lambda_perceptual * perceptual_loss_fn(derained, clean)
    total_loss += p_loss
    postfix["perceptual_loss"] = f"{p_loss.item():.4f}"

if args.use_contrastive_loss :
    p_loss = args.lambda_contrastive * contrastive_loss_fn(derained, clean, rainy)
    total_loss += p_loss
    postfix["contrastive_loss"] = f"{p_loss.item():.4f}"
loop.set_postfix(postfix)
```

最后在主函数中传参即可。

```
contrastive_loss_fn = EasyContrastiveLoss().to(device) if
args.use_contrastive_loss else None

train_one_epoch(train_loader, model, optimizer, pixel_loss_fn, perceptual_loss_fn,
contrastive_loss_fn, args, device)
```

2.使用 `AdaptiveAvgPool2d` ，可以将每个通道的 $H \times W$ 的特征图求平均值，最终得到一个 $B \times C \times 1 \times 1$ 特征图，我们将他用 `flatten` 拉扁就能得到一个可以反映图像的“高阶语义”的向量，计算损失时，我们有当前的预测值 `pred` 、目标的无雨图 `target` 以及原始的有雨图 `rainy` ，为使预测值接近无雨图且“远离”有雨图，我们希望 `pred` 与 `target` 的余弦相似度尽可能接近1，且与 `rainy` 的余弦相似度尽可能地接近-1，因此我们定义 $loss = \cos(\text{pred}, \text{rainy}) - \cos(\text{pred}, \text{target})$ ，这样一来，损失越小，说明预测图象越接近无雨图、越“远离”有雨图。

代码实现：

```
import torch as th
import torch.nn as nn
import torch.nn.functional as F
from torchvision.models import vgg19, VGG19_Weights

class EasyContrastiveLoss(nn.Module) :
  def __init__(self) :
    super(EasyContrastiveLoss, self).__init__()
    vgg = vgg19(weights = VGG19_Weights.IMAGENET1K_V1).features
    self.slice = vgg[:35]
    self.slice.eval()
    for p in self.slice.parameters() :
      p.requires_grad = False
    self.pool = nn.AdaptiveAvgPool2d(1)
    self.mean = th.tensor([0.485, 0.456, 0.406]).view(1, 3, 1, 1)
    self.std = th.tensor([0.229, 0.224, 0.225]).view(1, 3, 1, 1)

  def get_vector(self, img) :
    img_norm = (img - self.mean) / self.std

    feat = self.slice(img_norm)
```

```
        v = self.pool(feat).flatten(1)

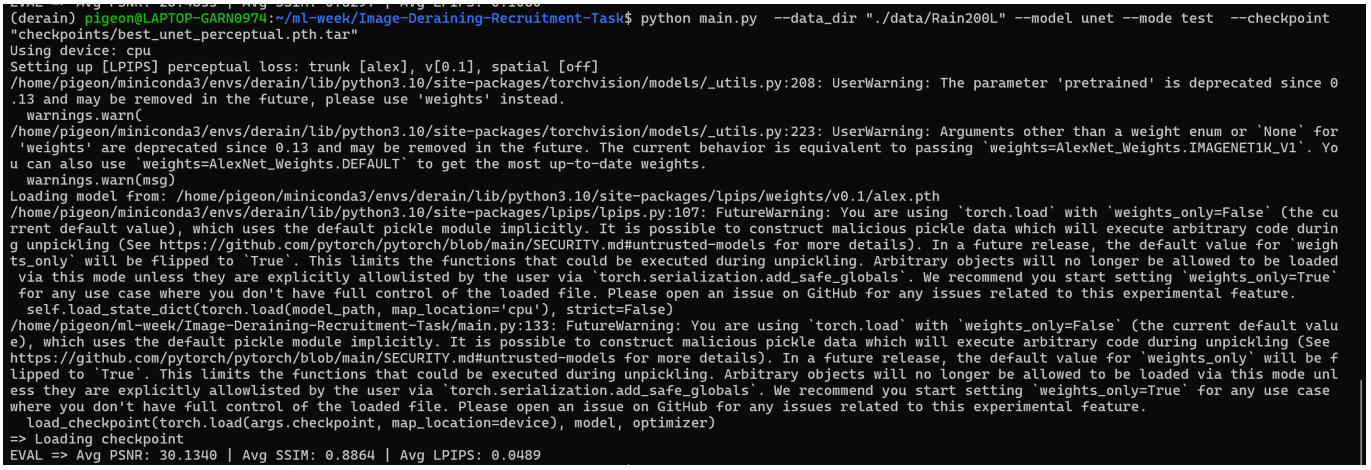
        return F.normalize(v, p = 2, dim = 1)

    def forward(self, pred, target, rainy) :
        v_pred = self.get_vector(pred)
        v_target = self.get_vector(target)
        v_rainy = self.get_vector(rainy)

        cos_target = (v_pred * v_target).sum(dim = 1).mean()
        cos_rainy = (v_pred * v_rainy).sum(dim = 1).mean()

        loss = cos_rainy - cos_target
        return loss
```

3.这次训练只为尝试新的损失函数，且题面提到不用过多关注指标高低，因此选择只训练十轮，测试结果：



附加2：我还是目标函数的端水高手😎

1.去雨任务的完成度可以从多方面进行量化评估，如像素损失、感知损失等，每种损失的算法不一样，因此数量级也不一样，如果直接把损失相加只会量纲失衡，最终得到的损失值将由数值较大的损失完全主导，因此需要给每种损失设置一个权重。

2.因为看似能提高模型性能且功能各不相同的目标函数之间可能本身是冲突的，选用多个目标函数加在一起可能会使模型找不到优化的方向，最终原地打转或者陷入一个劣解出不来。

3.可以尝试对损失值进行标准化，保存每一轮训练的损失值，每次拿当前损失值和过去损失值计算出来的标准差和平均数去给当前损失值做标准化，这样就把每种损失控制在了同一个数量级，此时超参数就不用用来平衡不同损失之间的量级，而是用于调控每个损失函数发挥的作用。